

WEEK3

NodeJS &JS Programming

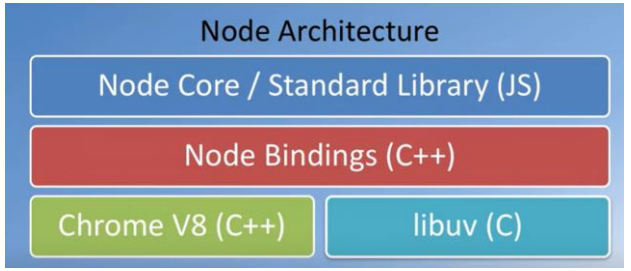
CN5006: Web and Mobile Application Development

Dr. Nadeem Qazi

Week 3 Node JS &JS Programming

- NodeJS
- Introduction to Node JS:NPM
- Java Script functions
- Arrow Functions
- Nodes Modules
- How to create Local Modules
- How to export the Modules data
- Synchronous vs Asynchronous

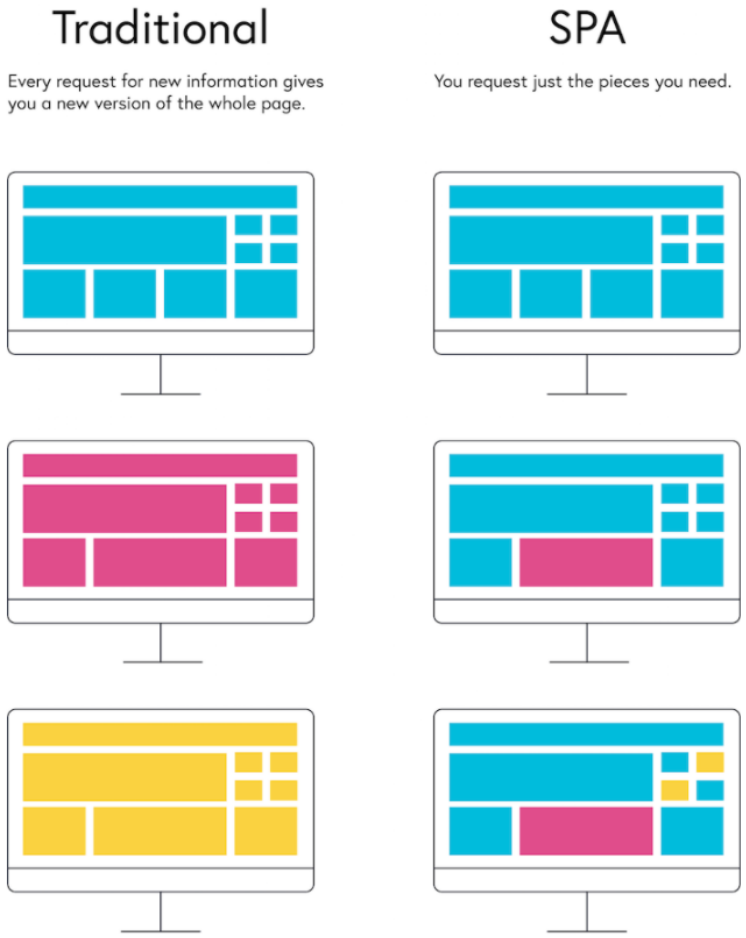
What is NodeJS



1. Node is a Server side JavaScript runtime environment built on Chrome's V8 JavaScript engine.
2. Allows developers to make different servers like DNS server, TCP server, HTTP server, and more through built in APIs
3. Node has an event-driven architecture, allows building scalable light weight real time web applications
4. Its unique asynchronous, non-blocking I/O model eliminates the waiting approach to serving requests.
5. Node's package manager npm, is the largest open source ecosystem in the world
6. **Apps Types: Single page Realtime DataStreaming, RestAPI Server side Rendered CL tools & HW Programming**

Node JS Application Types

- Single-page apps (SPAs): The single page application is a web application or website that interacts with the user by dynamically rewriting the current page, rather than loading entire new pages from the server.
- SPA are different than a traditional page load where the server re-renders a full page with every click you make and sends it to your browser.
- The SPA only sends what you need with each click, and your browser renders that information.
- SPAs include social networking apps, email or map apps, online text or drawing tools, etc.



University of East London

Node JS Application Type

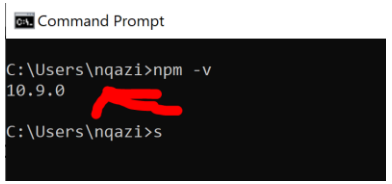
- Real-time apps (RTAs): Web apps that enable users to receive information as soon as it's published by an author example include instant messaging apps or chat rooms, video conference apps, etc.
- Data streaming apps: These are apps that send data/content as it arrives while keeping the connection open to continue downloading further content. video- & audio-streaming apps.
- REST APIs: Are interfaces that provide data for someone other app. For example, a Facebook API for Posts.
- Server-side rendered apps (SSRs): These web apps can run on both the client (front-end) and the server (back-end) allowing pages that are dynamic to display HTML. SSRs utilize SPA methods in that they don't need to reload every time you use it.
- Command line tools: These allow you to automate repetitive tasks and then distribute your tool across the vast Node.js ecosystem. An example of a command line tool is cURL, which stand for client URL and is used to download content from an internet URL.
- Hardware programming: Node.js can enable data collection, analysing that data, communicating back and forth between a device and server and taking action based on the analysis. IoT uses, collecting data from sensors.
- NPM contains more than 80 packages for Arduino controllers, raspberry pi,

  Intel IoT Edison, various sensors, and Bluetooth devices.

University of East London

Node Package Manager (npm)

- The Node Package Manager is the manager for the Node ecosystem and automatically gets installed during the installation of the Nodes
- Used to install publicly available packages on NodesJS,
- When using npm, you will most likely use the command line tool for most of your interactions.
- For better understanding please watch another video on NPM.
- **To see if NPM is installed, type npm -v in Terminal. This should print the version number:**



- **A typical Node package consists of JavaScript files together with a file called package.json which is the manifest file for this node module.**

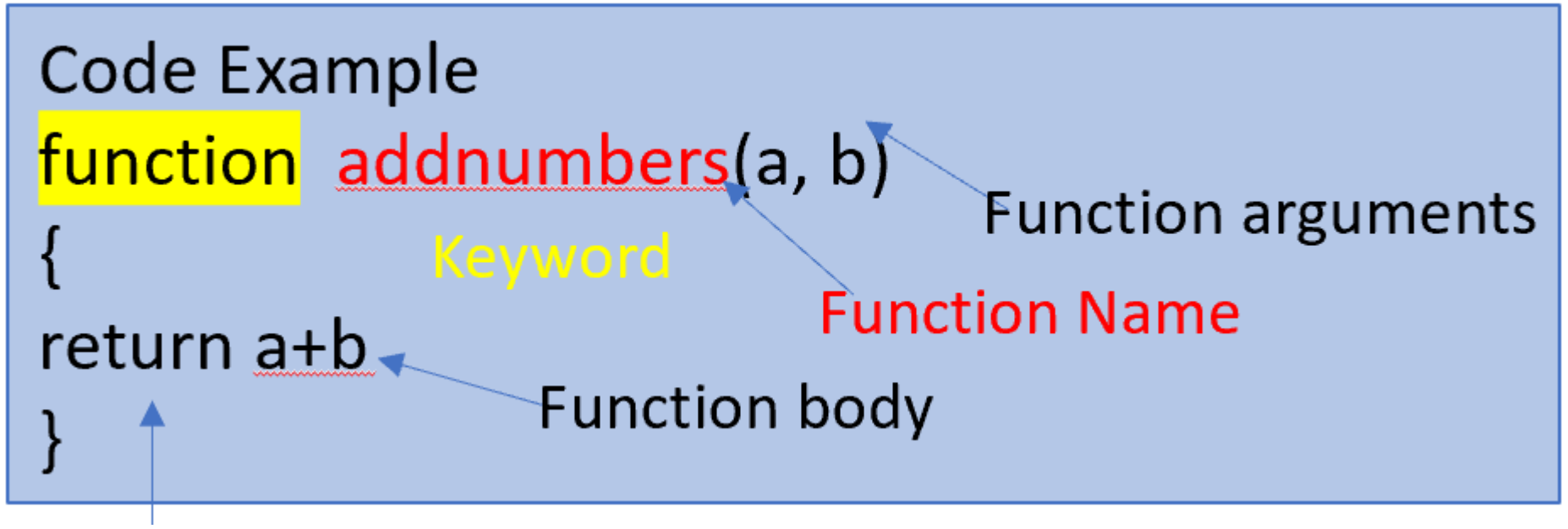
express	React	react-dom	mongodb & mongoos
a de-facto web development framework for Node.js,	React is a JavaScript library for building user interfaces.	React package for working with the DOM.	MongoDB wrappers to provide the API for MongoDB object databases in Node.js.

For Installation please follow the guide MERNINSTALLATION.PDF

University of East London

Java Script Function

- A function is a subprogram designed to perform a particular task
  - ✓Functions are executed when they are called.
  - ✓Values can be passed into functions and used within the function.
  - ✓Functions return a value. In JavaScript, if no return value is specified, the function will return undefined
  - ✓ JS function is also a object. because they can have properties and methods .



University of East London

JS function

```
// Function declaration
function add(a, b) {
    return a + b;
}

// Function expression
const sum = function (a, b) {
    return a + b;
}

add(2, 3); // 5
sum(2, 3); // 5
```


University of East London

Java Script Functions

- Arrow Functions
  - arrow functions are short syntax, anonymous functions introduced in ES6
  - the arguments to the function are enclosed in brackets
  - body of the function is defined after => .

```
// ES5                    Function Decleration
var add = function(x, y) {
    return x + y
};

// ES6                    Function Expression
let add = function(x, y) => x + y

let add = (x, y) => x + y
```

- In case for non argument function it should be written as

```
ArrowFunction
addnumbers= function (a,b)=>{
return a+b
}
```

Note there is no function name

University of East London

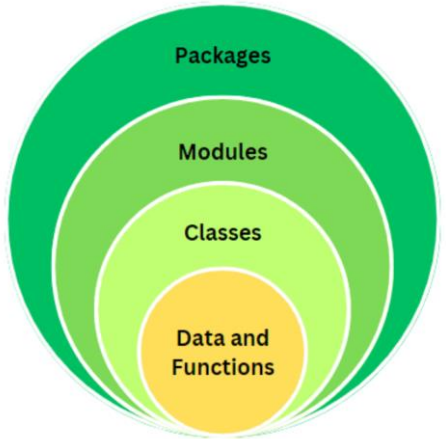
Nodes Module

- A module is a separate file that contains variables, function/s, object defined in a separate JavaScript file)
- In the Node.js module system, each file is treated as a separate module. Modules help us encapsulate our code into manageable chunks.
- Anything defined in the module (i.e. in JavaScript file) remains limited to that module only, unless we want to expose it to other parts of our code.

Module Types

- Core Modules:
  - Modules that come shipped with Node.js (see the table)
- Local Modules:
  - User defined Modules i.e. created by us. These modules basically consist of core business logic of our code.
- Third Party Modules
  - Modules that you install from any package manager i.e. npm . For example, express to create Web framework, mongo dB to access mongo dB etc
- What is package in Node JS
- A package is a directory with one or more modules inside of it. and a package.json file which has metadata about the package. A package must contain a package.json file in order to be published to the npm registry.
- Note:Since modules are not required to have a package.json file, not all modules are packages. Only modules that have a package.json file are also packages.
- https://nodejs.org/api/modules.html

Core Module	Description
http	http module includes classes, methods and events to create Node.js http server.
url	url module includes methods for URL resolution and parsing.
querystring	querystring module includes methods to deal with query string.
path	path module includes methods to deal with file paths.
fs	fs module includes classes, methods, and events to work with file I/O.
os	Provides information about the operating system
util	util module includes utility functions useful for programmers.
readline	To handle readable streams one line at the time



East London

Example 1: using os Core Module

- create a index.js file on the VSC and write the following code:

```
os=require("os")
const util=require('util')
console.log("temporary directory"+ os.tmpdir() )
console.log("hostname: "+ os.hostname())
console.log("OS : " + os.platform() +"release:"+ os.release())
console.log("Uptime"+  (os.uptime())/3600 +"  hours")
console.log("userInfo" + util.inspect(os.userInfo()))
console.log("Memory "+ os.totalmem()/1000000000 + "Giga byte")
console.log(" free: "+os.freemem()/1000000000 + "Giga byte")
console.log("CPU "+  util.inspect(os.cpus()))
console.log("Network"+ util.inspect(os.networkInterfaces()))
```

Save index.js and run you will see following output at the terminal

```
temporary directory/tmp
hostname: b5d96a55eee4
OS : linuxrelease:5.4.0-1019-gcp
Uptime2.9563888888888887  hours
userInfo{
  uid: 1000,
  gid: 1000,
  username: 'runner',
  homedir: '/home/runner',
  shell: '/bin/bash'
}
Memory 27.335294976Giga byte
 free: 12.111179776Giga byte
CPU [
  {
    model: 'Intel(R) Xeon(R) CPU @ 2.30GHz',
    speed: 2300,
    times: { user: 2039560, nice: 31020, sys: 972580, idle: 7223200, irq: 0 }
  },
  {
    model: 'Intel(R) Xeon(R) CPU @ 2.30GHz',
    speed: 2300,
    times: { user: 2072510, nice: 50670, sys: 966720, idle: 7188420, irq: 0 }
  },
  {
    model: 'Intel(R) Xeon(R) CPU @ 2.30GHz',
    speed: 2300,
    times: { user: 1988150, nice: 29310, sys: 973400, idle: 7251210, irq: 0 }
  },
```

Local Modules

- A module is a separate file that contains variables, function/s, object defined in a separate JavaScript file)
- Every things defined in the modules remains private to that module only, unless we want to expose it to other modules through using ‘export ’ or module.export
- Modules can be reused throughout the Node. JS- application.
- For example, if you need to connect to MongoDB and fetch data then you can create a module for it, which can be reused in your application.
- You can also package it and distribute it via NPM, so that Node.js community can use it

How to use Module in NodeJS

- To use either Core,Local or third party Module we use “require”function to import the module. For example suppose we want to use the core module “os ” in our Node JS programme, then the first we have to import this using osvrb=require(“os”)
- Some functions of the core module os

Name of the core module in this case it os. Suppose if we want to use core module fs then we will use fs

Method	Description
arch()	Returns the operating system CPU architecture
constants	Returns an object containing the operating system's constants for process signals, error cotes etc.
cpus()	Returns an array containing information about the computer's CPUs
endianness()	Returns the endianness of the CPU
EOL	Returns the end-of-line marker for the current operating system
freemem()	Returns the number of free memory of the system
hostname()	Returns the hostname of the operating system
loadavg()	Returns an array containing the load averages, (1, 5, and 15 minutes)
networkInterfaces()	Returns the network interfaces that has a network address
platform()	Returns information about the operating system's platform
release()	Returns information about the operating system's release
tmpdir()	Returns the operating system's default directory for temporary files
totalmem()	Returns the number of total memory of the system
type()	Returns the name of the operating system
uptime()	Returns the uptime of the operating system, in seconds
userInfo()	Returns information about the current user

University of East London

How to create Local Module

- Step 1: For any Local module ,create a separate file name it appropriate name
- Step 2: Create Any variable, function, object as per your logic. Note however anything defined in a module (i.e. in separate JavaScript file) **remains private to that** module only, unless we want to expose it to other modules through using 'export ' or module.export
- Takeaway: Export or module.export is used to make the content of the Module as public.

Example 2: How to create Local Module

Exposing both variable and function outside a Module

Create a file and name it as StudentInfo.JS write following code into it

Declare two Arrow functions and one constant variable

```
const getStudentName =  () =>
{
return 'The student name is ABC'
}
const getCampusName = () =>
{
return ("UEL Campus ")
}
```

```
// exporting  functions/ variables
const dateofBirth= "12/12/1980"
exports.getName=getStudentName
exports.Location=getCampusName
exports.dob=dateofBirth
```

```
// how to export function with parameters
exports.Studentgrade=(marks)=>{
if (marks>50 && marks <70)
return ("B grade")
else
return ("A grade)")
}
```


University of East London

Example : How to use Local Module

Use StudentInfo.Js in index.js
Require command is used for using the module in another file.

```
const student= require('./StudentInfo')
// because getName is the function  so we should use ()
console.log("Student Name:" +student.getName())
console.log(student.Location())
// because dob is not the function rather a variable so we do nt need to use ()
console.log(`${student.dob}`)
console.log(student.Studentgrade())
console.log("grade is "+student.Studentgrade(55) )
```

Exporting a Default Value

- when you have a module that exports just the one thing, it's more common to use module.exports ,Save following code into separate file names as Person.js
- Index.js

```
const jim = new User('Jim', 37, 'jim@example.com');

console.log(jim.getUserStats());
```

- Class exercise: create local module to find
- Area and perimeter of a rectangle
- Area= length*breadth
- Perimeter = 2*(length+breadth)

```
class User {
    constructor(name, age, email) {
      this.name = name;
      this.age = age;
      this.email = email;
    }

    getUserStats() {
      return `
        Name: ${this.name}
        Age: ${this.age}
        Email: ${this.email}
      `;
    }
  }
  module.exports = User;
```

Another Example

- Example
- create a separate module name it rectangle.js.
  - It will hold formulas for the perimeter and area of the rectangle. Perimeter =2*(length+breadth)and Area is length*breadth. We will use arrow functions to create these two functions in the separate file rectanglejs and then use export to make it visible to outside the moduel
- We will use rectangle.js in index.js using require function.

University of East London

Module Examples using Arrow Function

Create a file Rectangle.js

```
exports.perimeter =  (x, y) => (2*(x+y));

exports.area = (x, y) => (x*y);
```

- use this module in index.js

```
var rect = require('./rectangle');

function solveRect(l,b) {
    console.log("Solving for rectangle with l = " + l + " and b = " + b);

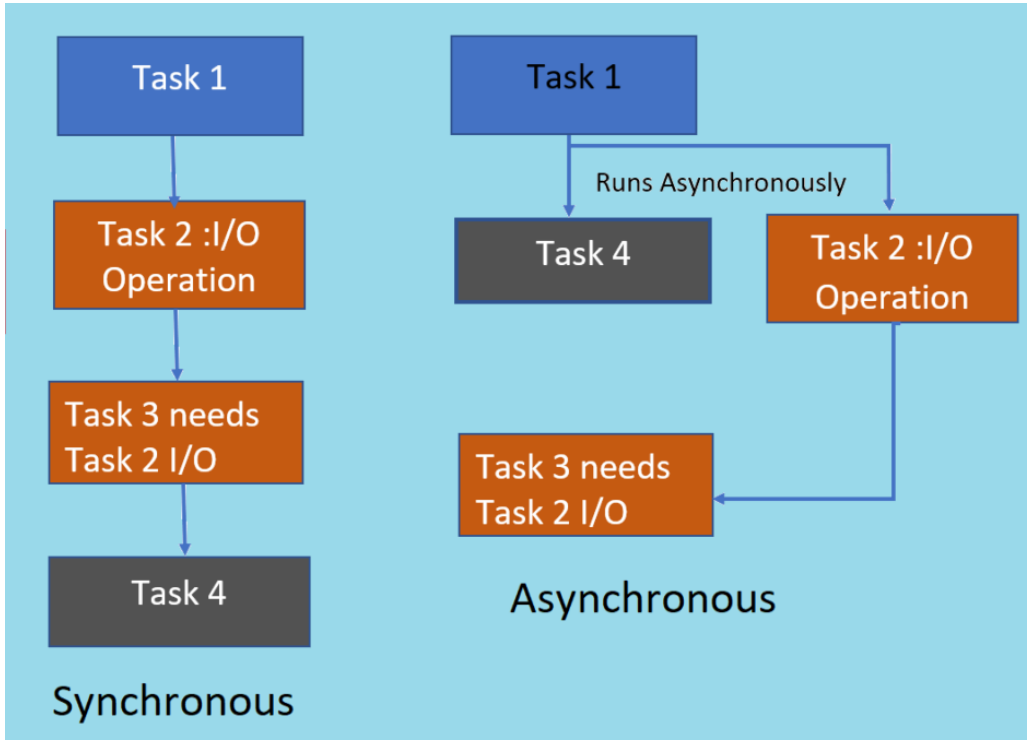
    if (l <= 0 || b <= 0) {
        console.log("Rectangle dimensions should be greater than zero:  l = "
                + l + ",  and b = " + b);
    }
    else {
        console.log("The area of the rectangle is " + rect.area(l,b));
        console.log("The perimeter of the rectangle is " + rect.perimeter(l,b));
    }
}

solveRect(3,4);
solveRect(3,5);
solveRect(0,5);
solveRect(-3,5);
```

University of East London

Synchronous vs Asynchronous

- Synchronous = happens at the same time.
  Asynchronous = doesn't happen at the same time
- Synchronous code execute in sequence. Each statement waits for the previous statement to finish before executing.
- Asynchronous (or async also called as nob blocking operation) execution refers to execution that doesn't run in the sequence that appears in the code. The program doesn't wait for the task to complete and can move on to the next task.

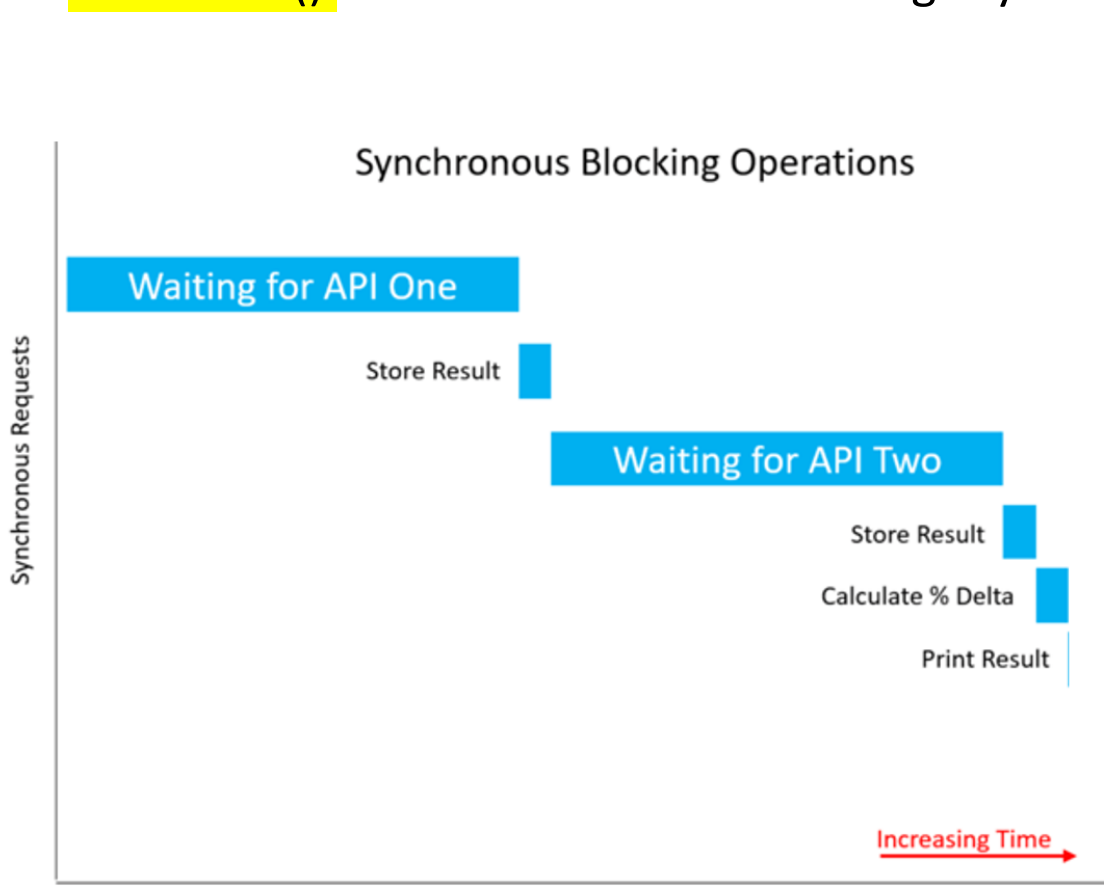


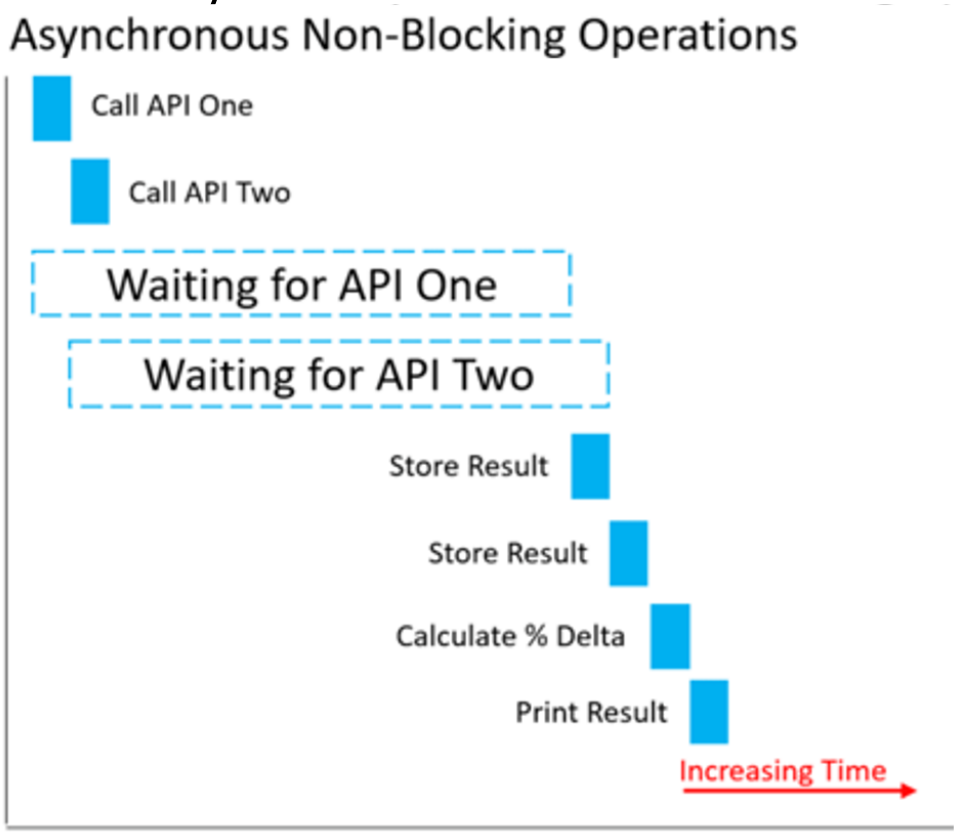
Node JS offers both synchronous and non synchronous file operation

University of East London

Example Read File(Synchronous & Asynchronous

- fs.readFileSync(): read files in a synchronous way, i.e node.js block other parallel process and do the current file reading process. the original node program stops executing, and node waits for the fs.readFileSync() function to get executed, after getting the result of the method the remaining node program is executed.
- fs.readFile() read a file in a non-blocking asynchronous way





University of East London

Reading file with synchronous and asynchronous

callback.js > ...

```
1  var fs = require('fs');
2
3  var data = fs.readFileSync('expressexample.js', {encoding:'utf8', flag:'r'});
4  console.log('Data: ', data);
5
6  console.log("reading second file")
7  data = fs.readFileSync('export.js', {encoding:'utf8', flag:'r'});
8  console.log('Data index.js: ', data);
9
10 // asynhronous
11 console.log("Reading file asyncronloly")
12 var data = fs.readFile('first.js', {encoding:'utf8', flag:'r'},function(err, data) { //Move on, this will fire when ready.
13   if(err) return console.log('Error: ', err);
14   console.log('Data: Async: ', data); // Assume var data is defined above.
15 });
16   data = fs.readFileSync('rectangle.js', {encoding:'utf8', flag:'r'});
17   console.log('readFileSync synchronus Data: ', data);
18 //process.exit(1)
```